Linked Lists

[CSE2010] Data Structures

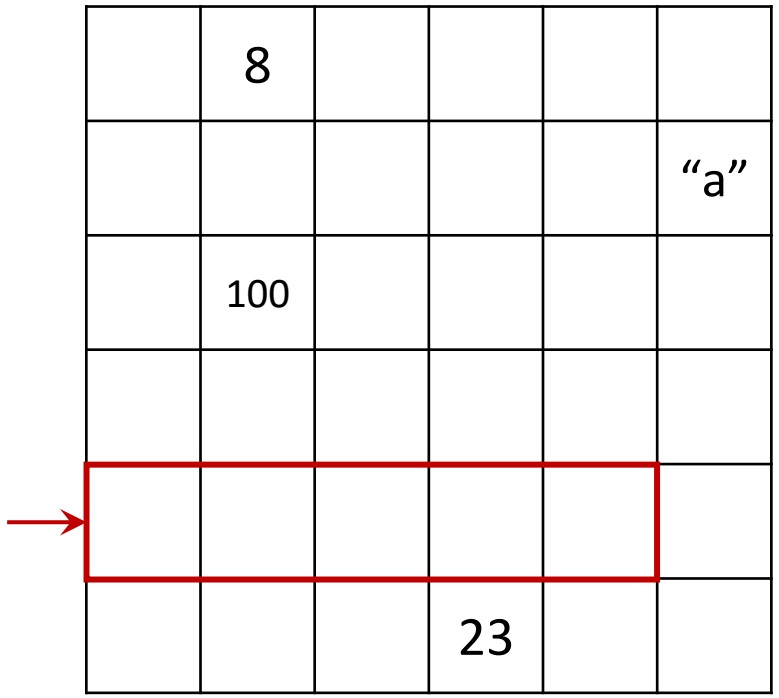
Department of Data Science

Linked Lists

Linked list is a collection of nodes that collectively form a linear sequence.

Memory

- When creating an array, your code finds a contiguous group of empty cells in memory and designates them to store data for your application.
- Computer can access any memory address in one step.
- You can immediately access any index within array.

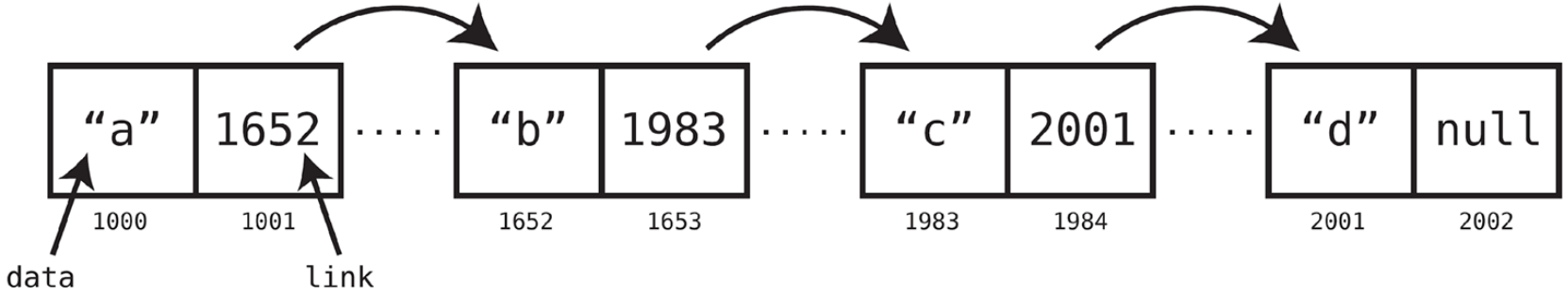


Array vs. Linked List

- An array is a contiguous block of memory.
- Data from a linked list can be scattered across different cells throughout the memory.

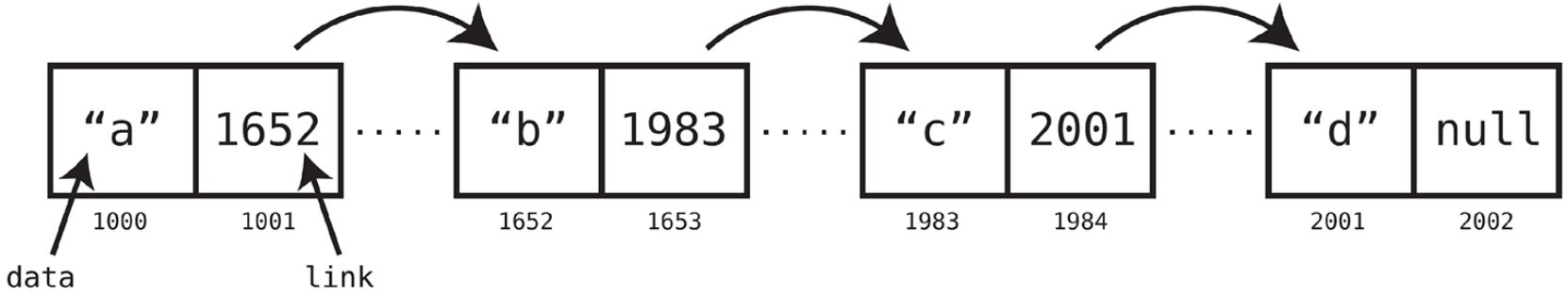
Linked Lists

- Connected data dispersed throughout memory are **nodes**.
- Each node represents one item in the list.
- How does the computer know which nodes are part of the same linked list?
  - Pointer to the next node's memory address (**link**)



Linked Lists

- First node is referred to as its **head**, and its final node as its **tail**.
- Successive elements are connected by pointers
- The last element points to NULL.
- Can grow or shrink in size during execution of program
- Can be made just as long as required.



Basic Operations in Linked List

- **Traversing** the linked list
- **Searching** for an item in the linked list
- **Inserting** a node to the linked list
- **Deleting** a node from the linked list

Traversing Linked List

- Follow the pointers
- Display the contents of the nodes as they are traversed.
- Stop when the next pointer points to NULL.

```
Traverse(L)
    x = L.head
    while x ≠ NULL
        print x.key
        x = x.next
```

Search

- Follow the pointers as long as the next pointer points to non-null and the key of current node is not the key to find

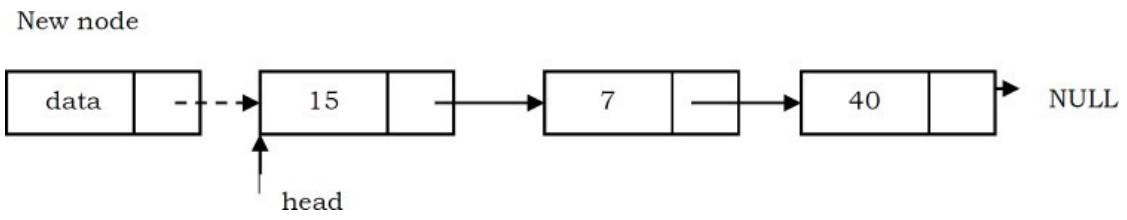
```
Search(L, k)
    x = L.head
    while x ≠ NULL and x.key ≠ k
        x = x.next
    return x
```

Insertion

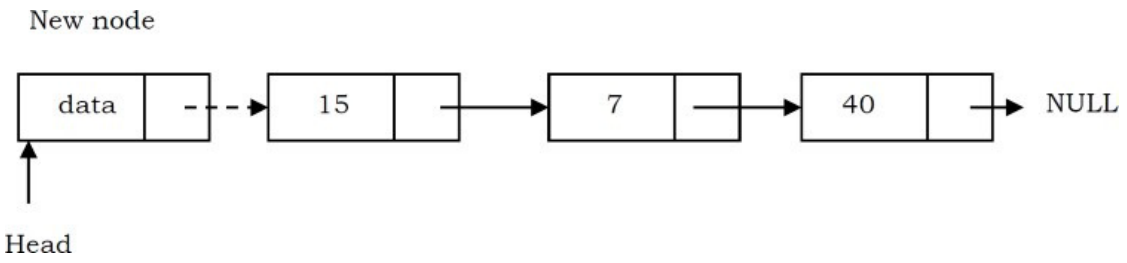
- Three cases
  - Inserting a new node before the head (at the beginning)
  - Inserting a new node after the tail (at the end)
  - Inserting a new node at the middle (random location)

Inserting a Node at the Beginning

1. Update the next pointer of new node, to point to the current head.



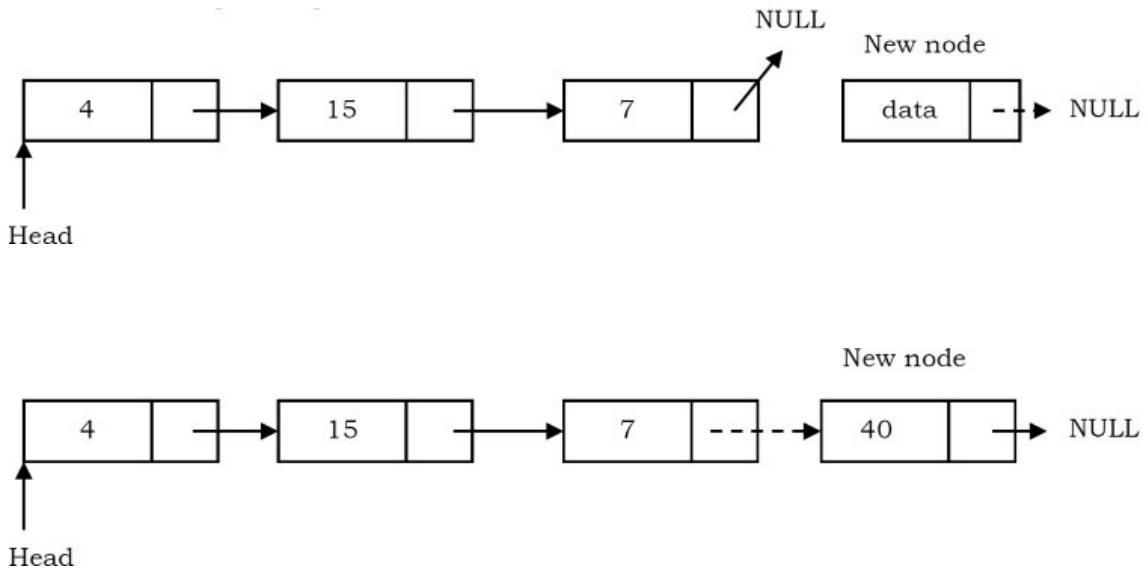
2. Update head pointer to point to the new node.



Inserting a Node at the End

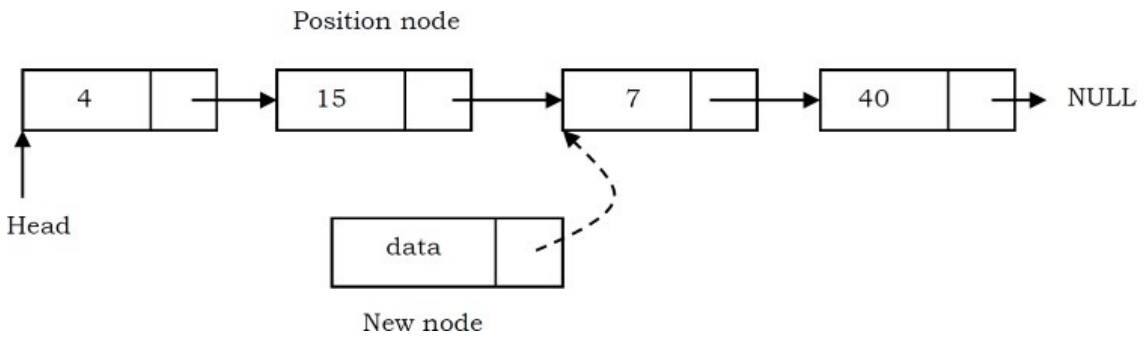
1. New node's next pointer points to NULL.

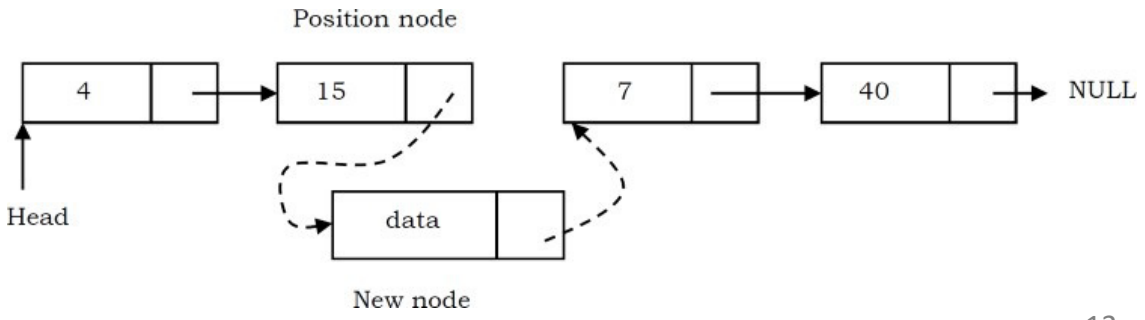
2. Last node's next pointer points to the new node.



Inserting a Node in the Middle

1. (If we want to add an element at position 3 then we stop at position 2. That means we traverse 2 nodes and insert the new node. Assume that the second node is called position node.) The new node points to the next node of the position where we want to add this node.
2. Position node's next pointer points to the new node.



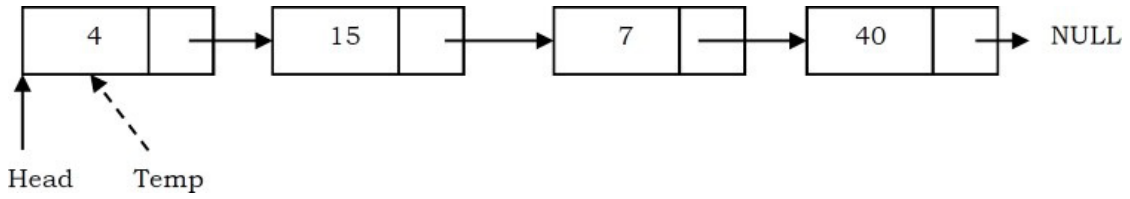


Deletion

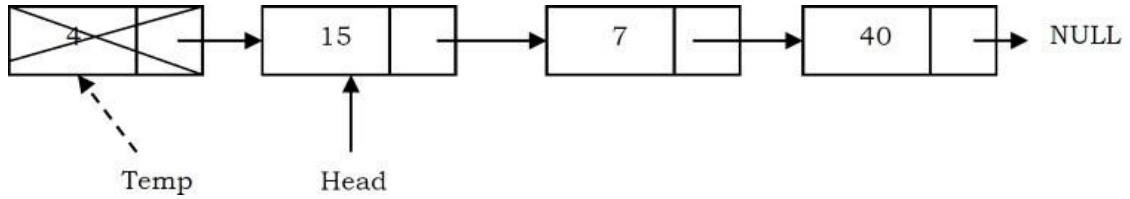
- Three cases
  - Deleting the first node
  - Deleting the last node
  - Deleting an intermediate node

Deleting the First Node

1. Create a temporary node that points to the same node as that of head.



2. Move the head nodes pointer to the next node and dispose of the temporary node.

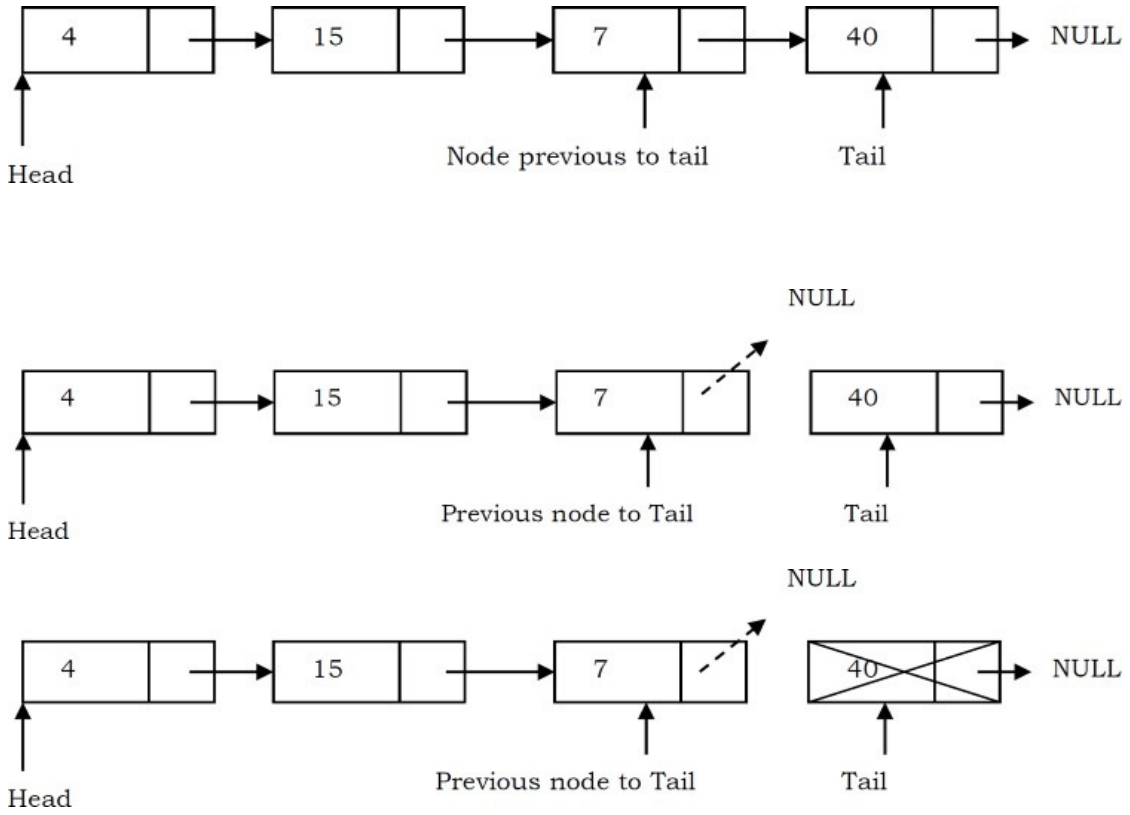


Deleting the Last Node

1. Traverse the list, and maintain the previous node while traversing

2. Update previous node's next pointer with NULL.

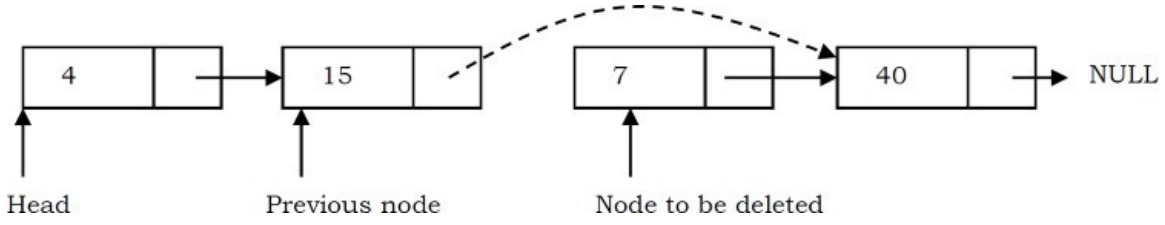
3. Dispose of the tail.

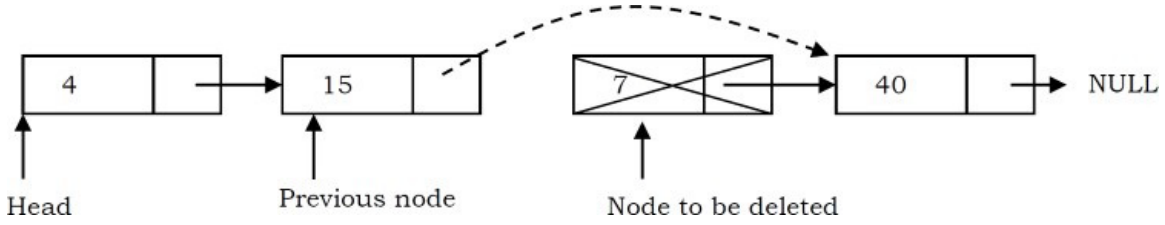


Deleting an Intermediate Node

1. Traverse the list until we find the node to be deleted. Maintain the previous node while traversing the list. Once we find the node to be deleted, change the previous node's next pointer to the next pointer of the node to be deleted.

2. Dispose of the current node to be deleted.





Arrays vs. Linked Lists

Advantages
Drawbacks

• Simple and easy to use
• Faster access to elements (constant access)
• Fixed size
  • Preallocate all needed memory up front
  • Waste memory space for unused space
• Expensive to insert an element
	• Hard to manipulate
	• Take O(n) for access to an element in the worst case
	• Waste memory in terms of extra reference points
	• Expanded in constant time

Time Complexity: Arrays vs. Linked Lists

Operation	Arrays	Linked Lists
Indexing (Get the i-th element)	O(1)	O(n)
Search	O(n)	O(n)
Insertion	O(n)	O(n)
Deletion	O(n)	O(n)

When do we use linked lists?

- Stacks, queues, hash tables, and graphs can be implemented using doubly linked lists.
- Most Recently Used (MRU) list
  - Linked list of file names
- Cache in web browser that allows you to hit the back button
  - Linked list of URLs
- Undo functionality in Photoshop or Word
  - Linked list of states

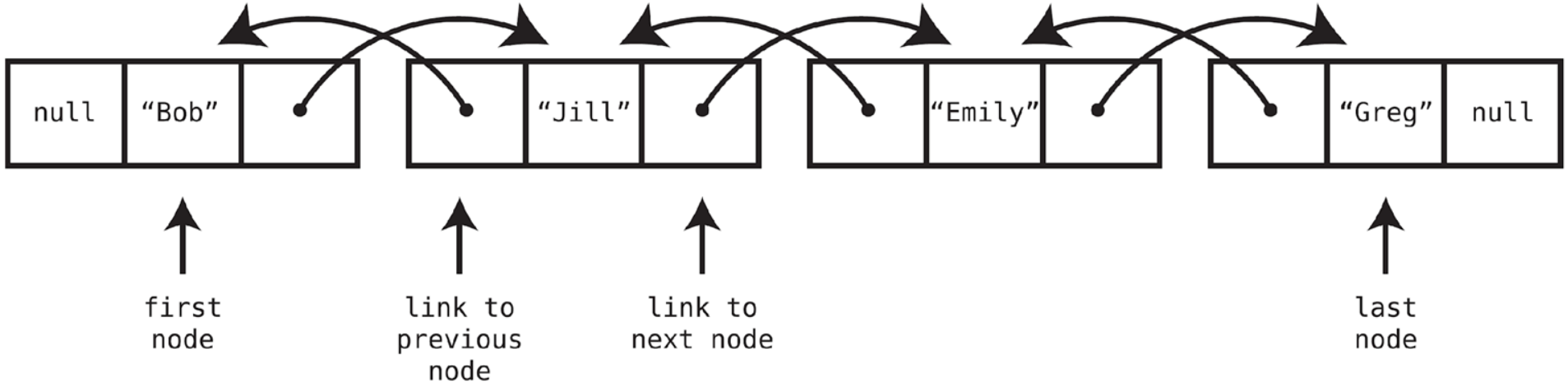
Doubly Linked Lists

Doubly linked list is like a linked list except that each node has two links – next and prev.

Doubly linked list always keeps track of both the first and last nodes, instead of just the first node.

Doubly Linked Lists

- Advantage
  - Given a node in the list, we can navigate in both direction.
  - We can delete a node even if we don't have the previous node's address
- Disadvantages
  - Each node requires an extra pointer, requiring more space.
  - The insertion or deletion of a node takes a bit longer (more pointer operations).

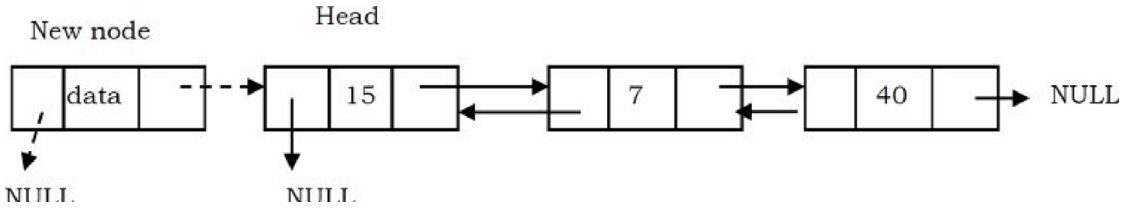


Insertion

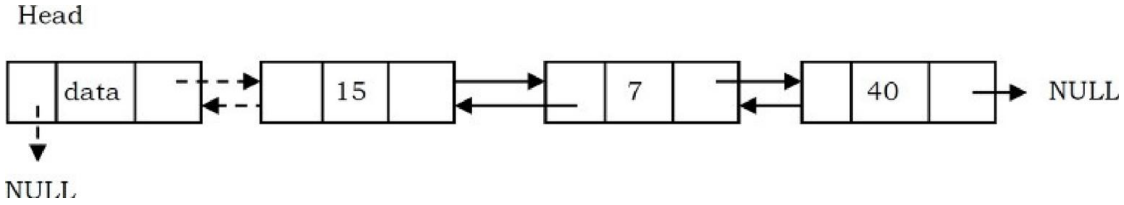
- Three cases
  - Inserting a new node before the head (at the beginning)
  - Inserting a new node after the tail (at the end)
  - Inserting a new node at the middle (random location)

Inserting a Node at the Beginning

1. Update the right pointer of the new node to point to the current head node (dotted link) and also make left pointer of new node as NULL.



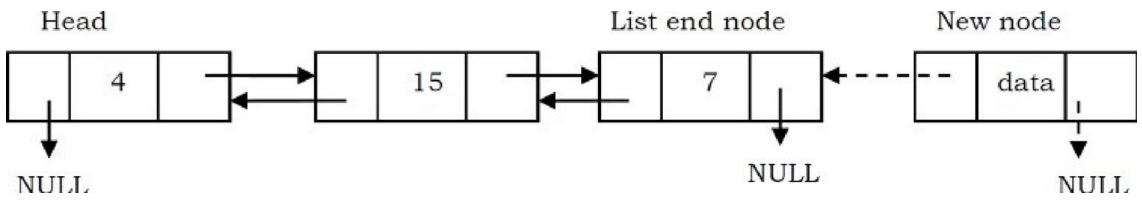
2. Update head node's left pointer to point to the new node and make new node as head.

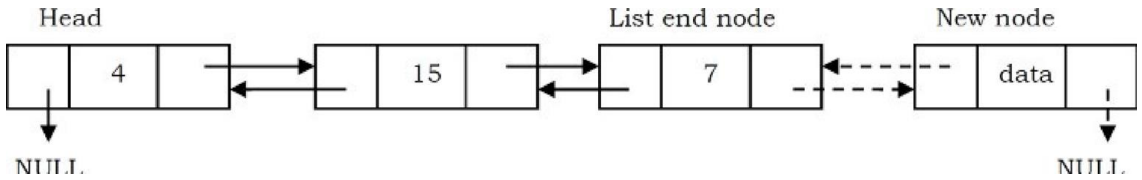


Inserting a Node at the End

1. New node's right pointer points to NULL and left pointer points to the end.

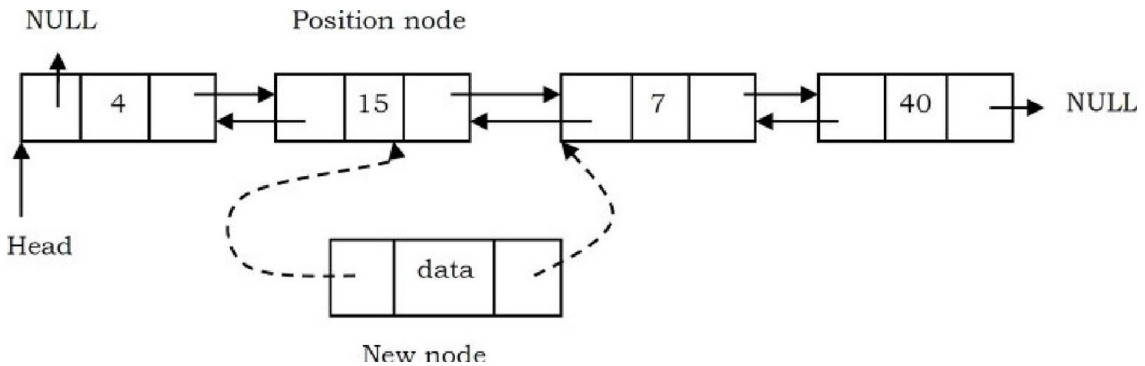
2. Update the right pointer of the last node to point to new node. Make new node as tail (if you defined the tail of the list).

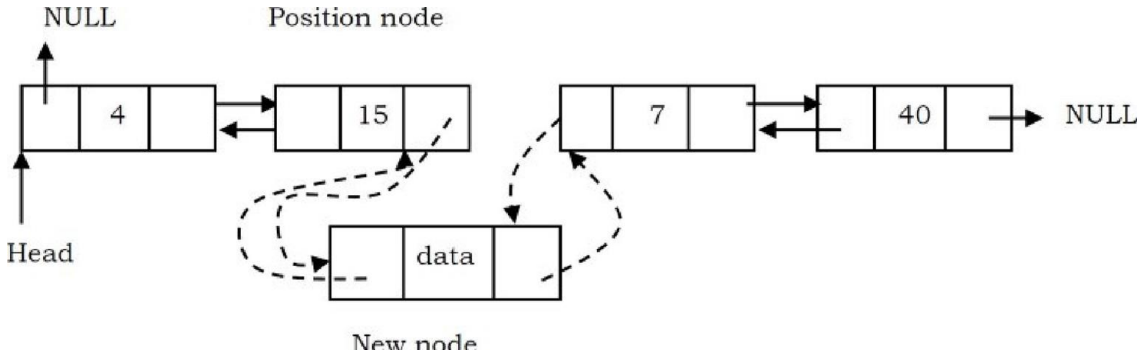




Inserting a Node at the Middle

1. New node's right pointer points to the next node of the position node where we want to insert the new node. Also, new node's left pointer points to the position node.
2. Position node's right pointer points to the new node, and the next node of position node's left pointer points to new node.





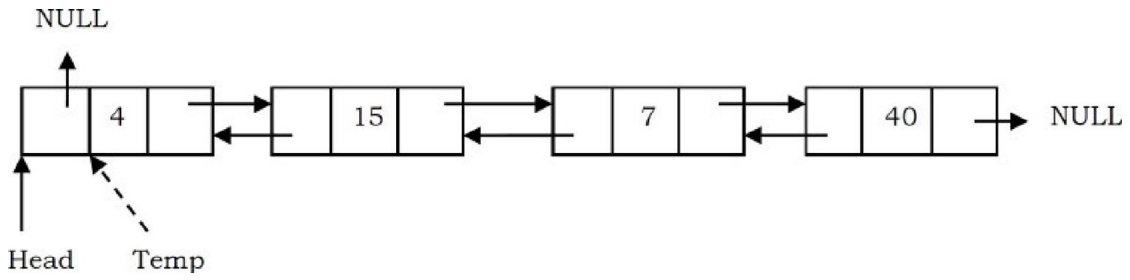
Deletion

- Three cases
  - Deleting the first node
  - Deleting the last node
  - Deleting an intermediate node

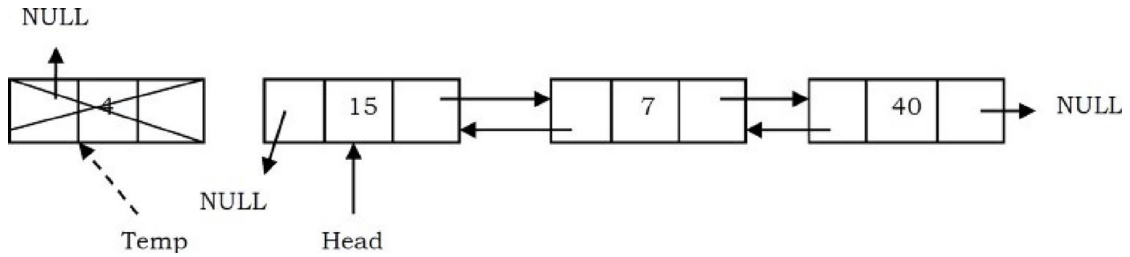
```
Delete (L, x)
   if x.prev ≠ NULL
      x.prev.next = x.next
   else L.head = x.next
   if x.next ≠ NULL
      x.next.prev = x.prev
```

Deleting the First Node

1. Create a temporary node which will point to the same node as that of head.



2. Move the head node's pointer to the next node and change the head's left pointer to NULL. Then, dispose of the temporary node.

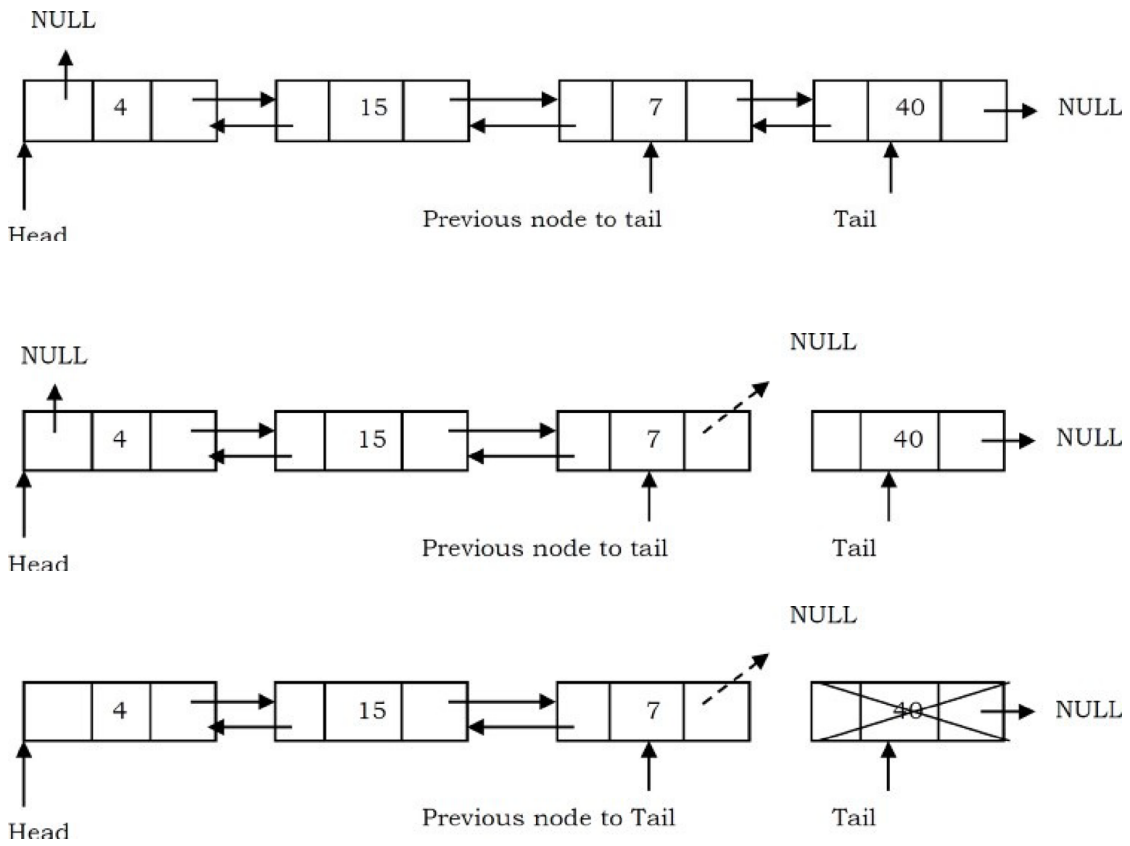


Deleting the Last Node

- (If the tail is not defined) traverse the list and while traversing maintain the previous node's address.

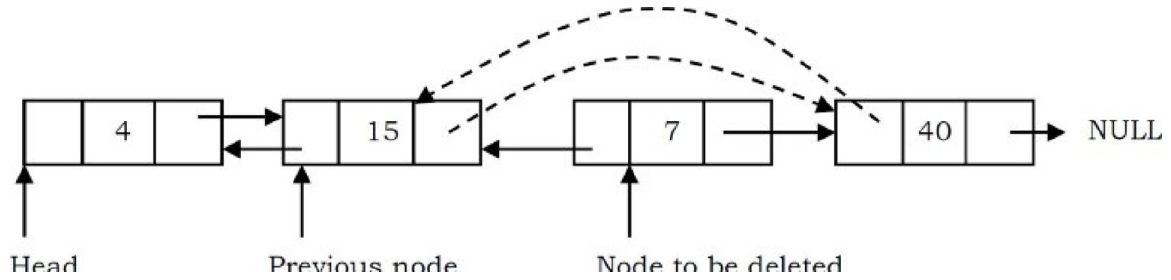
1. Update the next pointer of previous node to the tail node with NULL.

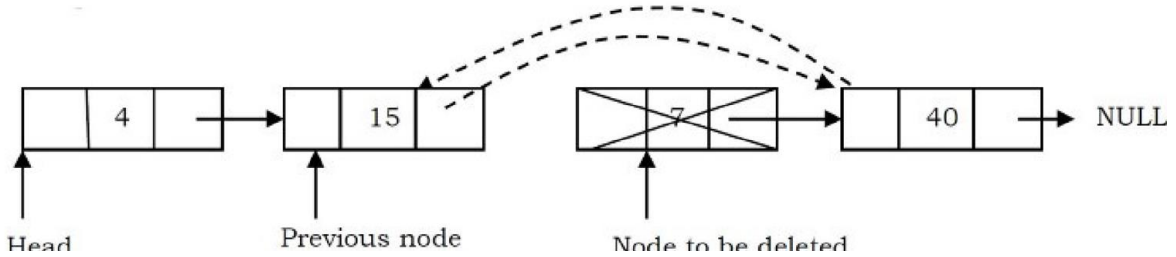
2. Dispose the tail node.



Deleting an Intermediate Node

1. Maintain the previous node while traversing the list. Upon locating the node to be deleted, change the previous node's next pointer to the next node of the node to be deleted.
2. Dispose of the current node to be deleted.





Wrapping up

- Subtle differences between arrays and linked lists
- Concept of nodes
- Linked list is only the simplest of node-based data structures